Canon Camera Control App — Project Kickoff

A wired-IP, Middle-Control-style camera control application for Canon cinema bodies, with Stream Deck integration via Bitfocus Companion. Prepared as the kickoff brief for Claude Code.

1. Project Summary

Build a desktop camera-control app for live production, similar in spirit to Middle Things' Middle Control, but for Canon cameras over a **fully wired IP network**. There is no gimbal / PTZ motion control in scope — this app handles **camera settings and record control only**, plus live view monitoring and touch focus.

Cameras (initial)

1. **Canon EOS C300 Mark III** — controlled via Canon XC Protocol over its built-in Ethernet port. The official XC Protocol documentation will be placed in `docs/` — Claude Code must read it fully before writing any client code.
2. **Canon EOS R5 C** — no official API (no XC Protocol, no CCAPI). It will be controlled via its **Browser Remote HTTP endpoints**, with the camera on the LAN through a USB-C-to-Ethernet adapter (officially supported by Canon in video mode). The endpoints are undocumented and will be captured with browser dev tools; build the R5 C module as a clearly separated driver with stubbed endpoint definitions to be filled in.

> Note: the R5 C connection is still HTTP-over-IP — the USB-C adapter just replaces Wi-Fi with a cable. It is not EDSDK-style USB tethering. Record the camera firmware versions in the repo, since the R5 C endpoints are the part most likely to change after a firmware update.

Network topology (all wired, zero RF)

C300 Mark III	built-in Ethernet	→	**Network switch** (single LAN)
EOS R5 C	USB-C → Ethernet adapter	→	
Control laptop + Companion + Stream Deck	Ethernet	→	

2. Core Requirements

Tech & architecture

• Electron app, or a local web app served by Node — Claude Code should recommend one and justify the choice. Clean dark UI suitable for dim production environments.

• A common **camera driver interface**, with `XCProtocolDriver` and `R5CBrowserRemoteDriver` implementations, so more cameras can be added later.

• The driver abstraction must **not assume HTTP**: a future driver may be backed by a native binding or a local sidecar process (see Sony, section 6).

• The app is the **single source of truth**: only the app talks to cameras. Companion and any other clients talk to the app's API, never to cameras directly (prevents state drift; the R5 C Browser Remote session is finicky with multiple clients).

Per-camera controls

- Record start/stop with clear REC state indication.
- ISO / gain, shutter, iris, white balance (kelvin + presets).
- ND filter control (C300 Mark III).

Global & supporting features

- "REC ALL" / "STOP ALL" across all connected cameras.
- Camera discovery/config: manual IP entry with saved camera profiles (JSON config file), connection status indicators, automatic reconnect on network drop.
- Presets: save/recall full camera-setting snapshots per camera.
- State sync: the UI must reflect the camera's actual current settings, including changes made on the camera body itself (polling or protocol events).
- Robustness: all network calls with timeouts and retries; the app must never lock up because a camera went offline.

3. Bitfocus Companion / Stream Deck Integration

Two-stage integration. Stage one ships with the core app; stage two is a proper Companion module.

Stage 1 — local REST API (works with Companion's Generic HTTP instance)

- The app runs a local REST API (and optionally an OSC listener) exposing every camera action: record start/stop per camera and globally, set ISO/shutter/iris/WB/ND, recall preset N.
- Predictable routes, e.g. `POST /api/cameras/:id/record/start` and `POST /api/presets/:id/recall`, so Companion's Generic HTTP instance can trigger them with no custom module.
- State exposure: `GET /api/cameras/:id/status` returning rec state and current settings, plus a WebSocket or SSE stream broadcasting state changes in real time.

Stage 2 — native Companion module

- Scaffold a proper Bitfocus Companion module (`companion-module-base`, TypeScript) in a separate package within this repo, patterned on existing open-source Companion modules.
- Actions for all camera controls; **feedbacks** for REC/tally state (button turns red while recording); **variables** for current ISO/shutter/iris/WB per camera.
- The REST API and the Companion module must share the same internal command layer — no duplicated camera logic.

4. Live View / Multiview

Each camera gets a video panel supporting multiple source types, selectable per camera in config:

- **(a) Protocol-native preview** — XC Protocol IP stream for the C300 III; Browser Remote live-view stream for the R5 C. Free fallback; lower quality/latency.
- **(b) Local capture device input** — UVC/USB capture cards via getUserMedia or ffmpeg, fed from the cameras' clean SDI/HDMI outputs. The high-quality path.

- **(c) NDI source** — plan the interface now; implementation can come later.
- **Decouple video from control:** a camera with no video source configured must still be fully controllable, and a dropped preview must never affect control or the REST API.
- **Multiview layout:** grid view of all cameras with REC tally borders (red outline when recording); click to focus a single camera full-panel with its controls.

Use clean feeds (camera SDI/HDMI clean output), not a program bus with graphics, especially when touch focus is in use (see next section).

5. Touch Focus on Any Video Source

Touch focus lives in the control protocol, not the video stream — so a tap on the SDI/capture feed can drive AF exactly like the native live view does. Requirements:

- Touch/click-to-focus must work on the video panel **regardless of video source type**. The tap is converted to normalized coordinates (0–1 in both axes, relative to the active image area, excluding letterbox bars) and sent via the camera's control protocol: XC Protocol AF frame position for the C300 III; Browser Remote touch-AF endpoint for the R5 C.
- Handle aspect-ratio mapping correctly: compute the displayed image rect within the panel and reject taps outside it.
- Per-camera calibration: optional X/Y offset and scale adjustment saved in the camera profile, for cases where the external feed framing differs slightly from the sensor's live-view coordinate space (crops, de-squeeze, switcher scaling).
- Visual feedback: draw the AF frame rectangle at the tapped position; if the protocol reports AF status, reflect it (e.g., green when focus locked).
- Expose "trigger AF at coordinates" and "one-shot AF" as REST API actions so Companion can trigger **focus presets** (e.g., Stream Deck buttons for stored positions: podium / drummer / wide).

Verify in the XC Protocol docs exactly which AF positioning commands the C300 III exposes (typically frame position + frame size) and promise only what the camera can do. Confirm the R5 C touch-AF endpoint empirically during the dev-tools capture session.

6. Future-Proofing: Sony Support

- Sony support comes later via Sony's **Camera Remote SDK** (native C++; covers FX-series and Alpha bodies; USB on all supported bodies, wired LAN on some cinema models).
- Do **not** implement Sony yet — just ensure the driver abstraction can be backed by a native binding or local sidecar process rather than only HTTP.

7. Build Plan (Phased)

Phase	Deliverable	Gate / proof
1	C300 III connect via XC Protocol: rec start/stop + ISO control	Proof of life — one camera, one wired link, REC toggles from the app
2	Full C300 III control set (shutter, iris, WB, ND) + presets + state sync	Settings changed on the body are reflected in the app

Phase	Deliverable	Gate / proof
3	REST API + WebSocket/SSE state stream; Companion via Generic HTTP	Stream Deck button starts/stops recording with no custom module
4	R5 C driver from captured Browser Remote endpoints	Rec + exposure control of R5 C over its USB-C Ethernet link
5	Live view multiview: protocol preview + capture-device sources, tally borders	Grid view with red REC outlines; preview drop does not affect control
6	Touch focus on all video sources + focus presets via API	Tap on SDI feed racks focus; Stream Deck recalls stored focus points
7	Native Companion module (actions, feedbacks, variables)	Button faces show REC state and current ISO

Sequencing rule: do not let live view creep into Phase 1. Control must be rock-solid first — it carries the hard external dependencies (protocol docs, endpoint sniffing). Video preview bolts on later with zero risk to working functionality.

8. Prerequisites & Owner Tasks (before / during build)

- Register with Canon's developer program (free) and obtain the **XC Protocol documentation**; place it in `docs/` before Phase 1 coding begins.
- R5 C endpoint capture: open Browser Remote in Chrome → dev tools → Network tab → press record / change ISO / tap focus on the web UI → save the captured requests (HAR export or copies) for Claude Code to turn into the driver.
- Hardware: USB-C-to-Ethernet adapter for the R5 C, network switch, Ethernet cables; later, capture devices or NDI encoders fed from clean SDI/HDMI outputs.
- Note current firmware versions of both cameras in the repo README.

9. Kickoff Prompt for Claude Code

Paste the following as the first message to Claude Code, run from the repo root with the XC Protocol PDF already in `docs/`:

```
I'm building a desktop camera-control app for live production, similar in spirit
to Middle Things' Middle Control, but for Canon cameras over a fully wired IP
network. No gimbal/PTZ motion control - camera settings, record control, live
view, and touch focus only.

CAMERAS
1. Canon C300 Mark III - controlled via Canon XC Protocol over Ethernet. The
   official protocol documentation is in docs/ - read it fully before writing
   any client code.
2. Canon EOS R5 C - no official API. Controlled via its Browser Remote HTTP
   endpoints (camera on the LAN via a USB-C Ethernet adapter). I'll capture
   the endpoints with browser dev tools; for now build the R5 C module as a
   clearly separated driver with stubbed endpoint definitions I can fill in.

CORE REQUIREMENTS
- Tech: Electron app (or local web app served by Node - recommend one and
  justify) with a clean dark UI suitable for a dim production environment.
- Architecture: a common "camera driver" interface, with XCProtocolDriver and
  R5CBrowserRemoteDriver implementations, so more cameras can be added later.
  The abstraction must NOT assume HTTP: a future driver may be backed by a
  native binding or local sidecar process (Sony Camera Remote SDK is planned
  later - do not implement it yet).
- Per-camera panel: record start/stop with clear REC state, ISO/gain, shutter,
  iris, white balance (kelvin + presets), ND filter (C300 III).
- Global controls: "REC ALL" / "STOP ALL" across all connected cameras.
- Camera config: manual IP entry with saved camera profiles (JSON config),
  connection status indicators, automatic reconnect on network drop.
- Presets: save/recall full camera-setting snapshots per camera.
- State sync: UI must reflect the camera's actual current settings, including
  changes made on the camera body itself.
- Robustness: all network calls with timeouts and retries; the app must never
  lock up because a camera went offline.
- The app is the single source of truth: only the app talks to cameras.
  External clients (Companion etc.) talk only to the app's API.

BITFOCUS COMPANION / STREAM DECK
- The app must run a local REST API (and optionally OSC listener) exposing
  every camera action: record start/stop per camera and globally, set
  ISO/shutter/iris/WB/ND, recall preset N. Use predictable routes like
  POST /api/cameras/:id/record/start and POST /api/presets/:id/recall so
  Companion's Generic HTTP instance can trigger them with no custom module.
- Expose state: GET /api/cameras/:id/status returning rec state and current
  settings, plus a WebSocket or SSE stream broadcasting state changes live.
- Phase 2 (after the core app works): scaffold a proper Bitfocus Companion
  module (companion-module-base, TypeScript) as a separate package in this
  repo: actions for all controls, feedbacks for REC/tally state (button turns
  red when recording), variables for current ISO/shutter/iris/WB per camera.
  Follow the structure of existing open-source Companion modules.
- The REST API and the Companion module must share the same internal command
  layer - no duplicated camera logic.

LIVE VIEW / MULTIVIEW
- Add a video panel per camera supporting multiple source types, selectable
  per camera in config:
```

```
  (a) protocol-native preview (XC Protocol IP stream for the C300 III,
      Browser Remote live view stream for the R5 C),
  (b) local capture device input (UVC/USB capture cards via getUserMedia or
      ffmpeg) fed from clean SDI/HDMI outputs,
  (c) NDI source (plan the interface; implementation can come later).
- Decouple video from control: a camera with no video source configured must
  still be fully controllable, and a dropped preview must never affect
  control or the REST API.
- Multiview layout: grid view of all cameras with REC tally borders (red
  outline when recording), click to focus a single camera full-panel.

TOUCH FOCUS ON ANY VIDEO SOURCE
- Touch/click-to-focus must work on the video panel regardless of source
  type. Convert the tap to normalized coordinates (0-1 both axes, relative to
  the active image area, excluding letterbox bars) and send via the control
  protocol: XC Protocol AF frame position for the C300 III, Browser Remote
  touch-AF endpoint for the R5 C.
- Handle aspect-ratio mapping correctly: compute the displayed image rect
  within the panel and reject taps outside it.
- Per-camera calibration: optional X/Y offset and scale adjustment saved in
  the camera profile, for when the external feed framing differs from the
  sensor's live-view coordinate space.
- Visual feedback: draw the AF frame at the tapped position; if the protocol
  reports AF status, reflect it (e.g., green when focus locked).
- Expose "trigger AF at coordinates" and "one-shot AF" as REST API actions so
  Companion can trigger stored focus presets from Stream Deck buttons.
- Verify in the XC Protocol docs exactly which AF positioning commands the
  C300 III exposes, and promise only what the camera can do.

PROCESS
Start by reading docs/, then propose the architecture and a phased build plan
(Phase 1: C300 III connect + rec + ISO as proof of life; live view and touch
focus come only after control is rock-solid). Ask me anything ambiguous
before writing code.
```

Tip: keep this PDF in the repo (e.g. docs/kickoff.pdf) so the full context survives between Claude Code sessions, and reference it in CLAUDE.md.